

Memoria — Fase 2

Parser de logs SDAP en tiempo real + visualización con Dash

De qué va el proyecto

El objetivo de la fase 2 era leer el log que genera log-sim.py (concretamente el fichero cu-lan-ho.log), extraer de cada línea cuántos bytes ha bajado cada UE, agregar esos bytes por segundo y pintar el resultado en una gráfica que se actualiza sola en una web local. El simulador va escribiendo nuevas líneas al log mientras corre, así que el parser tiene que comportarse como un tail -f y procesar los datos a medida que aparecen, sin parar y sin perderse ninguna línea.

Dicho así parece un trabajo rápido, pero cuando me puse a ello empezaron a aparecer detalles que no había tenido en cuenta al principio: el fichero no existe hasta que arranca el simulador, los eventos traen microsegundos pero yo quiero agregar por segundos, el último segundo de datos nunca se cerraba, algunos UEs desaparecían de la gráfica cuando dejaban de transmitir, y un largo etcétera. En esta memoria voy explicando, más o menos en el orden en que fui construyendo las cosas, cómo planteé cada pieza, qué problemas fueron saliendo y cómo los resolví.

Primer planteamiento

Lo primero que apunté en una libreta fue una versión muy simple: "un bucle que lee líneas del log, las parsea, suma bytes y actualiza la gráfica". Enseguida me di cuenta de que esa idea no iba a funcionar tal cual por tres motivos distintos:

- El fichero cu-lan-ho.log no existe hasta que el simulador se arranca. Si intento abrirlo antes me salta una excepción, así que el programa debe esperar a que aparezca en vez de fallar.
- Si en el mismo bucle leo el log y redibujo la gráfica, mientras Dash está ocupado pintando se me acumulan líneas sin leer. Y al revés: mientras leo el log y proceso datos, la interfaz se queda congelada.
- Los eventos vienen con marcas de tiempo con microsegundos, pero la gráfica muestra "bytes por UE y por segundo". Por tanto tengo que acumular los eventos del mismo segundo antes de pintar nada.

De esos tres puntos salió la arquitectura final, que tiene tres tareas asíncronas independientes (un productor, un consumidor y un watchdog) más la aplicación de Dash. Las tres tareas se comunican a través de una cola, y Dash corre en un hilo aparte para no bloquear el resto.

Patrón productor-consumidor con asyncio

Elegí asyncio porque permite tener varias "tareas" corriendo a la vez sobre un único hilo, sin tener que meterme con threads y locks. Lo bueno de asyncio es que cuando una tarea está esperando (por ejemplo, a que llegue una nueva línea del log) se suspende y le cede el control a otra tarea, así el programa aprovecha los huecos en vez de bloquearse.

async def y await, en corto

Todas las tareas del programa están definidas con async def en lugar de def. Escribir async def X(): no define una función normal, sino una corrutina: si la llamas como X() no se ejecuta, simplemente devuelve un objeto corrutina. Para que se ejecute de verdad hay que entregársela al event loop, ya sea con await o con asyncio.create_task.

Dentro de una corrutina, cada vez que aparece `await algo`, Python suspende esa tarea en ese punto hasta que `algo` esté listo. Mientras tanto, el event loop aprovecha para ejecutar otras corrutinas que sí estén listas. Es importante tenerlo claro porque significa que el `await asyncio.sleep(0.5)` que aparece en el productor **no bloquea** al resto del programa: simplemente dice "vuelve a mí dentro de medio segundo", y mientras tanto el consumidor y el watchdog siguen trabajando.

El error típico que conviene NO cometer

Si en cualquier parte del código asíncrono hubiese puesto `time.sleep(0.5)` en vez de `await asyncio.sleep(0.5)`, el programa entero se habría quedado parado durante ese medio segundo, porque `time.sleep` es síncrono y no le devuelve el control al event loop. Es un bug muy difícil de ver leyendo el código porque las dos líneas parecen equivalentes, pero no lo son en absoluto. Lo mismo pasa con hacer una petición HTTP con `requests.get` (síncrono) en lugar de con `aiohttp` (asíncrono): el primero bloquea, el segundo no.

asyncio.create_task y asyncio.run

En la función `main` lanzo las tres tareas con `asyncio.create_task(...)`. Esa llamada planifica la corrutina para que se ejecute pero **no espera** a que termine, sigue adelante con la siguiente línea. Así consigo que productor, consumidor y watchdog arranquen los tres y vayan corriendo en paralelo. Si hubiese hecho `await tail_file_producer(...)` en lugar de `create_task`, el programa se quedaría esperando a que terminase el productor (que no termina nunca, porque hace un bucle infinito) y nunca arrancarían ni el consumidor ni el watchdog.

La función `main` es a su vez una corrutina, así que para arrancar el programa hago `asyncio.run(main())` al final del fichero. Eso crea el event loop, ejecuta `main` y cuando termina cierra el loop.

La cola como canal de comunicación

La comunicación entre el productor y el consumidor se hace por una `asyncio.Queue`. El productor no conoce al consumidor, y viceversa: simplemente la cola está ahí en medio. Esto es básicamente el patrón productor-consumidor clásico, pero con colas asíncronas en vez de colas de threads.

```
queue = asyncio.Queue()

await data_queue.put(event) # productor mete un evento
item = await data_queue.get() # consumidor se queda esperando
```

Las dos operaciones, `put` y `get`, se usan con `await`. En el caso de `get` eso es clave: si la cola está vacía, el `await` suspende al consumidor y no lo reanuda hasta que aparezca un elemento. No hay busy-waiting, no hay polling, no se desperdicia CPU. La idea la saqué de la documentación oficial de Python (docs.python.org/3/library/asyncio-queue.html), donde el ejemplo es muy parecido al que acabé usando.

Dash en un hilo aparte

Hay un problema al mezclar Dash con asyncio: Dash por dentro arranca un servidor Flask, que es síncrono y bloqueante. Si llamase `app.run(...)` directamente dentro de `main`, el event loop se quedaría atascado ahí y las tres corrutinas no se ejecutarían. La solución es meter esa llamada en un hilo separado con `asyncio.to_thread`:

```
await asyncio.to_thread(
```

```

    app.run,
    host="127.0.0.1",
    port=8059,
    debug=False,
)

```

De esa forma Flask corre en un hilo aparte y asyncio sigue repartiendo tiempo entre las tres tareas en el hilo principal, sin pisarse.

Parseando las líneas con expresiones regulares

Las líneas del log que me interesan son las de tráfico SDAP de bajada. Una línea típica tiene esta pinta (abrevio):

```
2026-01-19T08:56:44.123456 [SDAP-DL] DL: ue=1 pdu_len=4096 ...
```

De esa línea necesito tres datos: el timestamp, el identificador del UE y la longitud del PDU (o sea, los bytes que ha bajado). Usé expresiones regulares porque me parecía la forma más limpia de extraer trozos concretos, y las compilé una sola vez al principio del fichero con `re.compile`. Compilar la regex cuando la vas a usar muchas veces es bastante más rápido que pasar el patrón en cada llamada, y en mi caso la lanzo contra cada línea del log, así que la diferencia se nota.

La regex del timestamp, trozo a trozo

La más interesante es la del timestamp, porque al verla por primera vez parece un jeroglífico, pero desmontándola por piezas es bastante razonable:

```

timestamp_re = re.compile(
    r"^(?P<ts>\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}\.\d+)"
)

```

`r"..."` — la `r` de delante convierte el string en un raw string. Sin ella tendría que escapar cada barra invertida ("`\\d`" en vez de "`\d`"), lo que vuelve el patrón muy incómodo de leer. En regex esto es prácticamente obligatorio.

`^` — ancla "al principio de la línea". Como en mi log el timestamp siempre está al principio, esto me garantiza que no pillo por error otra cadena con pinta de fecha que pudiera aparecer más adelante.

`(?P<ts>...)` — es un grupo de captura **con nombre**. Todo lo que haya dentro del paréntesis se captura bajo la etiqueta `ts`, y luego lo recupero con `match.group("ts")`. Existen también grupos sin nombre `(...)` y luego `.group(1)`, pero con nombre el código se lee mucho mejor, sobre todo cuando tienes varios grupos distintos y no quieres acordarte de qué número es cada uno.

`\d{4}` — exactamente 4 dígitos (el año). La llave con un número indica "este número exacto de repeticiones". Si quisiera "entre 2 y 4 dígitos" sería `\d{2,4}`.

`-\d{2}` — un guion literal seguido de 2 dígitos (el mes). Lo mismo para el día.

`T` — la letra `T` literal. Es el separador que usa el formato ISO 8601 entre fecha y hora.

`\d{2}:\d{2}:\d{2}` — horas, minutos y segundos, siempre dos dígitos cada uno separados por dos puntos.

`\.\d+` — un punto literal (el `\.` hay que escapar con la barra porque un punto a secas en regex significa "cualquier carácter") seguido de uno o más dígitos. El `+` significa "uno o más". Si pusiera `*` sería "cero o más", que también valdría pero aceptaría timestamps sin microsegundos.

Una vez que el match funciona, con `datetime.fromisoformat(m_ts.group("ts"))` convierto el string capturado en un objeto `datetime` real de Python, que ya sí me sirve para ordenar, comparar y agrupar.

Las otras dos regex, con word boundaries

Las del UE y la longitud del PDU son más cortas pero introducen una construcción nueva:

```
ue_re = re.compile(r"\bue=(?P<ue>\d+)\b")
pdu_len_re = re.compile(r"\bpdu_len=(?P<pdu_len>\d+)\b")
```

`\b` — es un **word boundary** (límite de palabra). No consume ningún carácter; lo único que hace es asegurar que justo en ese punto hay una transición entre un carácter de palabra (letras, dígitos, guion bajo) y uno que no lo es (espacios, paréntesis, principio o final de línea). Con eso evito que al buscar `ue=` me pille accidentalmente una subcadena dentro de otra palabra, como si hubiera un `xue=5` por ahí perdido. En este log no creo que pase, pero me quedé más tranquilo añadiéndolo.

`\d+` — uno o más dígitos. Lo mismo que en el timestamp, combinado con un grupo con nombre para poder recuperar el valor luego.

Por qué `re.search` y no `re.match`

Usé `re.search` en todas las regex aunque algunos ejemplos por internet usan `re.match`. La diferencia es que `match` solo intenta encajar la regex al principio de la cadena, mientras que `search` la busca en cualquier posición. Como yo tengo el `^` dentro de la regex del timestamp, en ese caso concreto `match` también valdría, pero para las otras dos (UE y PDU), que pueden aparecer en medio de la línea, necesito sí o sí `search`. Por consistencia usé `search` en todas.

Truco barato para no lanzar la regex cuando no hace falta

Antes de aplicar las tres regex a una línea hago cuatro comprobaciones tontas con el operador `in`:

```
if "[SDAP" not in line: return None
if "DL:" not in line: return None
if "ue=" not in line: return None
if "pdu_len=" not in line: return None
```

Esto es mucho más rápido que lanzar `re.search` de entrada, porque el operador `in` sobre un string es una búsqueda de subcadena implementada directamente en C con algoritmos muy afinados. Si alguna de las cuatro palabras clave no aparece en la línea, descarto la línea casi instantáneamente. Solo las líneas que pasan los cuatro filtros pasan por las regex, que es la parte "cara" en términos de CPU. Cuando el log tiene muchas líneas que no son SDAP (y suele pasar), este truco reduce mucho el trabajo.

Cosas de Polars que me tocó aprender

Para almacenar y agregar los datos usé Polars en lugar de pandas. Polars es bastante más rápido, usa menos memoria y la sintaxis con expresiones me parece muy cómoda una vez le pillas el truco. Pero al principio algunas cosas me sorprendieron viniendo de pandas.

Crear DataFrames vacíos con schema

Al principio del fichero declaro dos DataFrames completamente vacíos, pero con el schema fijado:

```
events_df = pl.DataFrame(
    schema={
        "timestamp": pl.Datetime,
        "timestamp_sec": pl.Datetime,
        "ue": pl.Int64,
        "bytes_dl": pl.Int64,
    }
)
```

Decirle el schema de entrada es importante porque más tarde voy concatenándole filas una a una, y si no le digo los tipos desde el principio, Polars los infiere de la primera fila que encuentre. Si la primera fila tuviera, por ejemplo, un `bytes_dl` pequeño podría inferir `Int32` y después fallar al concatenar una fila con un `Int64`. Fijando el schema desde el inicio me curo en salud.

pl.col, .alias y las expresiones

Una cosa que me chocó de Polars frente a pandas es que no manipulas columnas directamente, sino que construyes "expresiones" con `pl.col()` que luego se aplican dentro de una operación:

```
second_agg = (
    second_df
    .group_by(["timestamp_sec", "ue"])
    .agg(pl.col("bytes_dl").sum().alias("bytes_dl"))
    .sort(["timestamp_sec", "ue"])
)
```

Leído de izquierda a derecha: agrupa por `timestamp_sec` y `ue`; dentro de cada grupo, toma la columna `bytes_dl` (`pl.col("bytes_dl")`), súmala (`.sum()`), y llama al resultado `"bytes_dl"` (`.alias("bytes_dl")`). Lo de `.alias` sirve para darle nombre a la columna resultante; si no, Polars le pondría un nombre autogenerado raro. Al final ordeno por `timestamp` y `UE` para que las filas queden consistentes.

pl.concat con vertical_relaxed

Uso `pl.concat(..., how="vertical_relaxed")` para ir añadiendo filas a los DataFrames globales. El "vertical" quiere decir "apila filas una debajo de otra" (frente a "horizontal" que concatena columnas). Lo de "relaxed" significa que es un poco más tolerante con los tipos: si una fila tiene `Int32` y otra `Int64`, en vez de fallar los promueve al tipo más amplio. Es útil porque cuando recibo eventos nuevos no siempre tengo garantizado que los tipos coincidan bit a bit.

Filtros, is_in y fill_null

En el callback de Dash uso un par de cosas más de Polars.

`pl.col("timestamp_sec").is_in(last_seconds)` crea un filtro booleano que es `True` en las filas cuyo `timestamp` esté en la lista `last_seconds`. Es básicamente el equivalente al `IN (...)` de SQL.

Y `fill_null(0)` sustituye los valores nulos de una columna por 0. Lo uso tras un `left join` contra la ventana de tiempo, para que los segundos en los que un UE no transmitió salgan como 0 en lugar de `null` (y así la gráfica no tenga huecos).

Por qué Polars y no pandas

Para este proyecto Polars me pareció mejor opción por tres motivos: es más rápido (especialmente en `group_by` con muchos grupos), consume menos memoria, y tiene

soporte nativo de parquet muy bien integrado. Pandas también haría el trabajo, pero en un programa que corre indefinidamente y va acumulando filas, la diferencia de rendimiento y de memoria importa.

Agregar eventos por segundo

Como los eventos vienen con resolución de microsegundos pero la gráfica enseña "bytes por UE y por segundo", hay que acumular los eventos que caen en el mismo segundo. El enfoque que seguí es:

- Mantengo una variable `current_log_second` con el segundo que estoy acumulando ahora.
- Y un buffer `current_second_buffer` con los eventos de ese segundo.
- Cuando llega un evento nuevo comparo su segundo contra `current_log_second` y tengo tres casos.

Los tres casos son:

- Mismo segundo que el actual → lo añadido al buffer y listo.
- Segundo posterior → cierro el segundo anterior (lo agrego a `agg_df` con el `group_by + sum`), vacío el buffer y abro el nuevo con este evento.
- Segundo anterior → el evento llegó tarde; lo ignoro para no romper el orden de los datos (esto no debería pasar normalmente, pero más vale curarse en salud).

Un pequeño truco: `replace(microsecond=0)`

Para quedarme solo con la parte del timestamp hasta los segundos (quitando los microsegundos) uso:

```
ts.replace(microsecond=0)
```

El método `.replace()` de `datetime` no modifica el objeto original (los datetimes son inmutables en Python), sino que devuelve uno nuevo con los campos que le pases reemplazados. Poniendo `microsecond=0` me quedo con el timestamp exacto al segundo. Es más limpio que tirar de cadenas o hacer operaciones raras, y me evita preocuparme por pérdidas de precisión.

El último segundo nunca se cerraba: watchdog

Cuando tenía todo más o menos funcionando descubrí un bug muy sutil: en la gráfica siempre faltaba el último segundo. Me llevó un rato dar con la causa, pero tiene todo el sentido una vez que lo ves. Un segundo se "cierra" (se agrega al DataFrame) solo cuando llega un evento del segundo siguiente. Pero si el log se queda parado un instante, o termina, ese último segundo se queda abierto para siempre y su barra nunca aparece.

La solución fue añadir una tercera corrutina, un watchdog, que cada 200 ms comprueba si ha pasado suficiente tiempo sin recibir eventos nuevos. Si sí, cierra el segundo abierto a la fuerza:

```
async def watchdog_task():
    while True:
        await asyncio.sleep(0.2)
        if (current_log_second is not None
            and len(current_second_buffer) > 0
            and last_event_wall_time is not None
            and (time.monotonic() - last_event_wall_time) >=
WATCHDOG_TIMEOUT_S):
```

```
finalize_current_second()
current_second_buffer = []
current_log_second     = None
```

time.monotonic() frente a time.time()

Un detalle importante es que uso `time.monotonic()` y no `time.time().time.time()` devuelve el tiempo del sistema (los segundos desde 1970), y si alguien cambia la hora del equipo mientras el programa está corriendo, o si el reloj del sistema se ajusta solo con NTP, puede saltar hacia atrás. Eso podría hacer que el watchdog piense que ha pasado un montón de tiempo cuando en realidad no, o al revés. `time.monotonic()`, en cambio, da un reloj que nunca retrocede y que no se ve afectado por cambios horarios; solo sirve para medir diferencias, que es justo lo que me hace falta aquí.

Por qué un reloj de pared y no el del log

Otra cosa importante es que no puedo usar el timestamp del log para decidir "llevamos un segundo sin recibir nada". El tiempo del log solo avanza cuando llega un evento; si no llega ninguno, ese tiempo se queda congelado y el watchdog nunca se dispararía. Por eso guardo `last_event_wall_time = time.monotonic()` cada vez que entra un evento, y el watchdog compara contra eso.

El valor de WATCHDOG_TIMEOUT_S

Puse el timeout en 1.2 segundos a base de prueba y error. Con 1 segundo justo a veces el watchdog se disparaba antes de que terminara el segundo actual y cerraba datos a medias. Con 1.2 segundos va sobrado; supongo que el log tiene pequeños huecos naturales entre bloques de eventos y 200 ms de margen son suficientes para no confundir "pausa breve" con "fin de segundo".

Guardando cada rato en parquet

Cada 30 segundos el consumidor guarda el DataFrame `agg_df` en un fichero parquet.

```
if now - last_parquet_save >= parquet_period_s:
    if agg_df.height > 0:
        agg_df.write_parquet(parquet_file)
    last_parquet_save = now
```

Elegí parquet por tres razones: es un formato columnar (muy eficiente para operaciones sobre columnas), comprime bastante bien comparado con CSV, y Polars lo lee y escribe de forma nativa y muy rápida. El propósito del volcado es básicamente tener un respaldo: si el programa se cae o tengo que reiniciarlo, al menos no pierdo el historial agregado.

La parte visual con Dash

Para la interfaz usé Dash, que es una librería que monta una pequeña web a partir de componentes de Python sin tener que escribir HTML ni JavaScript. Ya la habíamos visto en clase y era la opción más directa para tener una gráfica que se actualice sola.

El layout

El layout es simple: un título, una línea de texto con información de depuración y la gráfica, más un componente invisible que dispara el refresco:

```
app.layout = html.Div([
    html.H3("Tráfico DL por UE"),
```

```

    html.Div(id="status-text"),
    dcc.Graph(id="live-graph"),
    dcc.Interval(id="interval", interval=1000, n_intervals=0),
])

```

El componente `dcc.Interval` es el que hace toda la magia. No se muestra, pero incrementa su contador `n_intervals` cada 1000 ms, y eso dispara el callback.

El decorador `@app.callback`

El corazón de la app es una función decorada con `@app.callback`:

```

@app.callback(
    Output("live-graph", "figure"),
    Output("status-text", "children"),
    Input("interval", "n_intervals")
)
def update_data(n):
    ...

```

Un decorador en Python (el `@algo` justo encima de una función) es una manera corta de decir "aplica esta función transformadora a la que viene después". En este caso, `@app.callback(...)` registra `update_data` en la app de Dash y le dice que esa función ha de llamarse cada vez que cambie el `Input` indicado (el `n_intervals` del `Interval`), y que lo que devuelva se usará para actualizar los `Output` (la propiedad `figure` del gráfico y la propiedad `children` del div de texto). Es un patrón declarativo bastante limpio: tú describes qué depende de qué, y Dash se encarga del resto por dentro.

Subplots y no un único gráfico

Para la gráfica elegí `make_subplots` con un subplot por UE, en lugar de una sola gráfica con todos apilados. La razón es que los tráficos de los distintos UEs pueden diferir mucho en magnitud: si uno baja 900 kB en un segundo y otro 15 kB, el pequeño se vuelve invisible cuando comparten la misma escala Y. Con un subplot por UE cada uno tiene su propia escala Y y se ven igual de bien independientemente del volumen.

hovertemplate y los placeholders

Para personalizar el tooltip que sale al pasar el ratón por una barra usé `hovertemplate`:

```

hovertemplate=(
    "Time: %{x}<br>"
    "Bytes DL: %{y}<br>"
    f"UE: {ue}"
    "<extra></extra>"
)

```

Aquí hay dos cosas mezcladas que conviene distinguir. Las llaves de Python `{ue}` (dentro de un f-string) se sustituyen cuando se construye el texto, en tiempo de Python. En cambio `%{x}` y `%{y}` no son f-strings: son placeholders de Plotly que se rellenan **en el navegador** con los valores X e Y del punto por el que estés pasando el ratón. Por eso van entre `%{...}` y no entre `{...}`. El `<br>` es simplemente un salto de línea HTML dentro del tooltip. Y `<extra></extra>` es un truquito para ocultar la "etiqueta extra" que Plotly añade por defecto con el nombre de la traza a la derecha.

Cosas que tuve que arreglar sobre la marcha

Aparte de los problemas gordos (el último segundo que no se cerraba, Dash bloqueando el event loop), hubo tres detalles de la gráfica que tuve que ir puliendo una vez la vi funcionando.

El UE que desaparecía

Al principio, cuando un UE dejaba de transmitir durante los 10 segundos visibles, el subplot entero desaparecía de la figura en vez de quedarse vacío. La razón era que yo calculaba la lista de UEs a partir del DataFrame ya filtrado a la ventana visible:

```
# versión incorrecta
ue_list = filtered_df.select("ue").unique()...
```

Si el UE no tenía datos en esa ventana, no aparecía en `ue_list`, y `make_subplots` se reconstruía sin él; por eso la fila desaparecía. Lo arreglé sacando la lista del `agg_df` completo, que contiene el histórico de todos los UEs que han transmitido alguna vez:

```
# versión corregida
ue_list = agg_df.select("ue").unique()...
```

Barras a 0 en los huecos

Incluso con lo anterior arreglado, cuando un UE no transmitía nada en toda la ventana el subplot seguía quedándose con el título "UE N" flotando y el área del gráfico totalmente en blanco, sin ejes. La solución fue rellenar los segundos sin datos con valor 0 mediante un `left join` contra la ventana completa de tiempo y un `fill_null(0)`:

```
window_df = pl.DataFrame({"timestamp_sec": last_seconds})
ue_df = (
    window_df
    .join(ue_df_raw.select(["timestamp_sec", "bytes_dl"]),
        on="timestamp_sec", how="left")
    .with_columns(pl.col("bytes_dl").fill_null(0))
)
```

El `how="left"` es importante: un `left join` conserva todas las filas del DataFrame de la izquierda (`window_df`) y añade las columnas del de la derecha cuando coinciden, rellenando con nulos cuando no. Así siempre tengo una fila por cada segundo de la ventana, y los que no tenían datos se rellenan con 0 después del `fill_null`. Resultado: siempre sale una barra por segundo, aunque sea de altura 0.

Los ticks de tiempo en todos los subplots

Por defecto, cuando se usa `shared_xaxes=True` en `make_subplots`, Plotly solo muestra las etiquetas del eje X en el subplot de abajo del todo (es un ahorro de espacio). Pero eso hacía que en los subplots de arriba no supieras a qué segundo correspondía cada barra. Lo solucioné forzando los ticks en cada subplot dentro del bucle:

```
fig.update_xaxes(showticklabels=True, row=i, col=1)
```

Dejo el título "tiempo" solo en el subplot de abajo para no repetirlo en cada fila.

Detalles pequeños de Python que aparecen por el código

Aparte de los temas grandes, en el código hay varias construcciones pequeñas que quizá no sean evidentes a primera vista. Las agrupo aquí para no tener que explicarlas entre medias.

global en las funciones

Dentro del consumidor y el watchdog aparecen líneas como:

```
global agg_df, current_second_buffer, current_log_second
```

Esas declaraciones `global` son necesarias porque en Python, si dentro de una función haces `agg_df = algo`, por defecto se entiende que estás creando una variable **local** con ese nombre, y la variable de fuera se queda intacta. Para decirle a Python "cuando escriba a `agg_df` dentro de esta función, refiérete a la de ámbito global", hay que declararlo explícitamente con `global`. Si solo lees la variable sin asignarle, no hace falta; solo es necesario cuando modificas la referencia.

f.seek(0, 2): saltar al final del fichero

Cuando abro el log, lo primero que hago es:

```
f.seek(0, 2)
```

El segundo argumento de `seek` es el "whence", o sea desde dónde cuenta el desplazamiento. 0 significa "desde el principio del fichero" (es el valor por defecto), 1 "desde la posición actual" y 2 "desde el final". Así que `f.seek(0, 2)` me deja el cursor 0 bytes antes del final, es decir, **en el final**. Eso es justo lo que quiero: cuando el programa arranca, no quiero reprocesar todo el log histórico, solo las líneas nuevas que vayan llegando a partir de ahora.

errors="ignore" al abrir el fichero

Al abrir el log paso `errors="ignore"`:

```
open(path, "r", encoding="utf-8", errors="ignore")
```

Esto le dice a Python que, si encuentra bytes que no son UTF-8 válido, los descarte silenciosamente en lugar de levantar una excepción. En logs reales aparecen a veces caracteres raros (sobre todo si hay binario mezclado o algún problema de codificación) y prefiero perder esos bytes a que el programa se caiga por una línea mala.

if __name__ == "__main__":

Al final del fichero aparece:

```
if __name__ == "__main__":
    asyncio.run(main())
```

Esta línea, típica de Python, solo ejecuta el código de dentro si el fichero se está corriendo directamente (`python parser.py`), no si alguien lo importa desde otro módulo. Para este proyecto en concreto no tiene una función crítica, pero es buena costumbre dejarlo ahí por si algún día reutilizo este fichero como módulo importable para tests o lo que sea.

f-strings por todo el código

Uso bastante el formato de f-string, que son strings prefijados con `f` y permiten meter expresiones de Python dentro de llaves. Por ejemplo:

```
print(f"✓ Closed log second {current_log_second} → {second_agg.height} rows added")
```

Dentro de las llaves puedo poner cualquier expresión, no solo nombres de variable, y también formatearla. Es la forma moderna y más legible de construir strings en Python (frente al `%` del estilo antiguo o el `.format()`).

Arrancando todo junto

La función main arranca las tres corrutinas con create_task y después llama a Dash en un hilo aparte:

```
async def main():
    asyncio.create_task(tail_file_producer(data_file, queue))
    asyncio.create_task(consumer(queue))
    asyncio.create_task(watchdog_task())

    await asyncio.to_thread(
        app.run,
        host="127.0.0.1",
        port=8059,
        debug=False,
    )
```

Los tres create_task lanzan las corrutinas en segundo plano y main sigue hasta el await asyncio.to_thread, que es el que se queda bloqueado mientras Dash corre en el hilo separado. Si ese await termina (porque cerrase Dash), toda la app se cerraría al acabar main.

Qué podría mejorarse

- Escribir tests unitarios del parser. Las regex las fui retocando a base de prints y eso no es muy robusto; un par de tests con líneas de ejemplo me habrían dado mucha más seguridad al cambiar cosas.
- Controlar el crecimiento de events_df. Ahora crece indefinidamente mientras el programa esté corriendo, y para ejecuciones largas eso consumiría cada vez más memoria. Podría vaciarlo cuando los datos ya se hayan agregado, o rotarlo a parquet y borrar de memoria.
- Añadir reconexión / reapertura del fichero si es rotado por el simulador. Actualmente si el log se borra y se vuelve a crear durante la ejecución, el programa sigue aferrado al file descriptor viejo.
- Manejar de forma más elegante el cierre del programa (Ctrl+C): cerrar el último segundo pendiente, volcar agg_df a parquet una última vez, etc. Ahora mismo al hacer Ctrl+C se corta en seco.
- Reemplazar el patrón de "regenerar la figura entera en cada tick" por actualizaciones incrementales (extendData de Plotly) cuando la ventana crezca mucho, para no redibujar datos que no han cambiado.

Conclusión

Lo que empezó siendo "leer un log y pintar una gráfica" acabó teniendo bastante más miga de la que esperaba: programación asíncrona con productor-consumidor y una cola, parseo con regex (más el truco del in para descartar líneas antes de lanzar la regex), agregación con Polars, persistencia en parquet, gráficas en vivo con Dash, un watchdog para casos raros y varios detalles de Plotly que solo descubrí al ver el resultado en pantalla.

Lo que más me costó fue entender por qué el último segundo no aparecía: el bug no daba ningún error, simplemente faltaba una barra en el gráfico, así que fue un rato de mirar el código sin pistas hasta que caí en que era un problema de cierre de estado. La solución con el watchdog quedó limpia y, de paso, me obligó a separar bien tres ámbitos de tiempo

distintos: el tiempo del log, el tiempo monotónico del sistema y el tiempo de refresco de Dash.

Si tuviera que empezar de cero, lo primero que haría sería montarme un par de tests unitarios del parser antes de ponerme con el resto. Por lo demás, el esqueleto de productor-consumidor con asyncio ha quedado bastante limpio y creo que lo reutilizaría como punto de partida para cualquier otro programa que tenga que procesar un stream de datos en tiempo real.